

补充章节：优化器演变与现代激活函数

前置阅读：本讲义第四章「梯度下降法与反向传播」和第六章「激活函数」。本章在此基础上深入，适合学完基础内容后拓展阅读。

第一部分：梯度下降法的演变

从最基础的 SGD 到 AdamW，优化器的发展始终围绕三个核心矛盾：

1. **收敛速度 vs 稳定性**：大步长收敛快但容易震荡
2. **全局一致 vs 逐参数自适应**：不同参数可能需要不同的学习率
3. **梯度方向 vs 历史方向**：当前梯度噪声大，历史梯度可提供惯性

1.1 起点：朴素 SGD

在第四章中，我们使用了最朴素的随机梯度下降（SGD）：

$$\theta_{t+1} = \theta_t - \eta \cdot g_t$$

其中 $g_t = \nabla_{\theta} \mathcal{L}(\theta_t)$ 为当前 mini-batch 的梯度， η 为学习率。

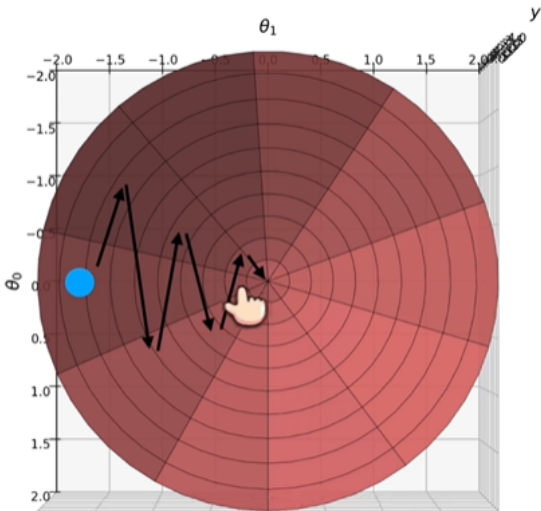
SGD 的三个痛点：

- 峡谷地形中反复震荡（某些方向陡峭、某些方向平缓）
- 所有参数共享一个学习率，不够灵活
- 纯靠当前梯度，噪声大

1.2 动量法（Momentum）

直觉

想象一个球从山坡滚下：它不仅有当前重力的作用（梯度），还有之前积累的速度（动量）。动量法在更新中引入“惯性”，使参数在一致的方向上加速，在震荡的方向上互相抵消。



公式推导

动量法维护一个速度向量 v_t ，它是指数衰减的梯度累积：

经典动量 (CM, Classical Momentum) :

$$\begin{aligned}v_t &= \mu \cdot v_{t-1} - \eta \cdot g_t \\ \theta_{t+1} &= \theta_t + v_t\end{aligned}$$

其中 $\mu \in [0, 1)$ 为动量系数（通常取 0.9）。

Nesterov 加速梯度 (NAG, Nesterov Accelerated Gradient) :

NAG 的核心改进是“**先看一步**”：先在动量方向上走一步，再在那个“前瞻”位置计算梯度，相当于做了更好的方向预估。

$$\begin{aligned}v_t &= \mu \cdot v_{t-1} - \eta \cdot \nabla_{\theta} \mathcal{L}(\theta_t + \mu \cdot v_{t-1}) \\ \theta_{t+1} &= \theta_t + v_t\end{aligned}$$

为什么 NAG 更好？ 在峡谷地形中，普通动量可能会因为惯性冲出最优区域再折返；NAG 提前在动量指向的位置评估梯度，从而及时“踩刹车”，减少超调。

展开形式 (理解动量的“记忆”)

将 v_t 递推展开：

$$\begin{aligned}v_t &= -\eta g_t + \mu v_{t-1} \\ &= -\eta g_t + \mu(-\eta g_{t-1} + \mu v_{t-2}) \\ &= -\eta \sum_{i=0}^t \mu^{t-i} g_i\end{aligned}$$

解读：当前速度是历史所有梯度的**指数加权移动平均**。越久远的梯度，权重 μ^{t-i} 越小。这解释了动量法为什么能够：

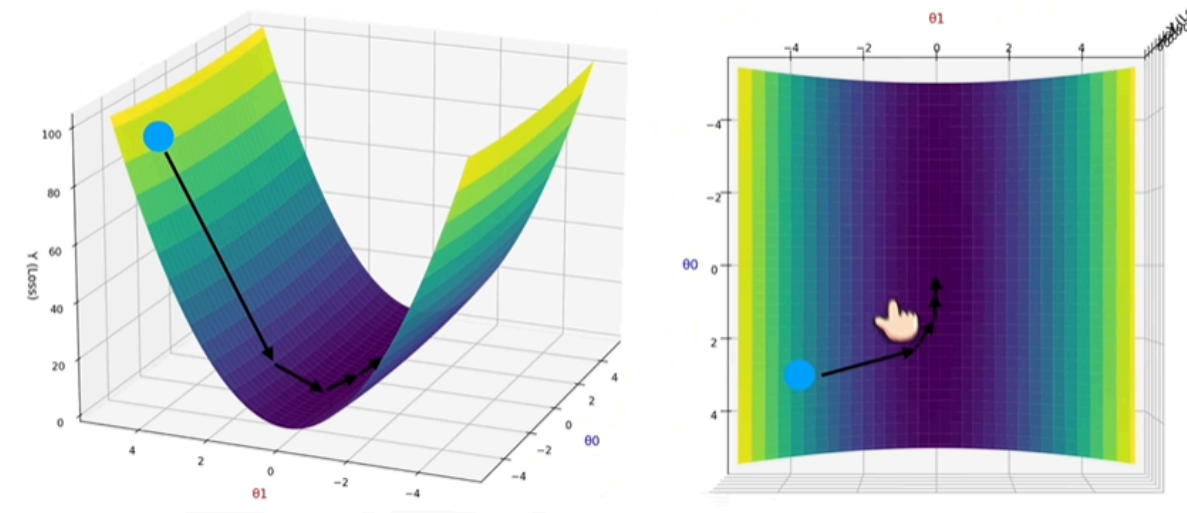
- **平滑噪声：**单一 mini-batch 的梯度噪声被历史平均消解
- **加速一致方向：**连续同向梯度 $\rightarrow v_t$ 持续增长 $\rightarrow$ 等效学习率变大
- **抑制震荡方向：**正负交替的梯度 $\rightarrow$ 加权平均接近零

1.3 RMSProp

动机

SGD 和动量法对所有参数使用统一的学习率。但在深度网络中，不同层的梯度尺度可能相差几个数量级——比如 embedding 层某参数梯度可能为 10^{-5} ，而输出层某参数梯度可能为 10^{-1} 。统一学习率会导致部分参数更新过猛，部分几乎不动。

RMSProp (Root Mean Square Propagation) 由 Hinton 在 Coursera 课程中提出，核心思想是**为每个参数自适应地调整学习率**：梯度大的参数 $\rightarrow$ 学习率自动缩小；梯度小的参数 $\rightarrow$ 学习率自动放大。



公式推导

RMSProp 维护梯度平方的指数移动平均 s_t :

$$s_t = \rho \cdot s_{t-1} + (1 - \rho) \cdot g_t^2$$

参数更新 (逐元素除法):

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{s_t + \epsilon}} \odot g_t$$

其中:

- ρ 为衰减率, 通常取 0.9 (Hinton 的建议) 或 0.999
- ϵ 为防止除零的小常数, 通常取 10^{-8}
- $\odot$ 表示逐元素乘法
- 所有向量运算 (平方、开方、除法) 均为逐元素操作

为什么用平方梯度的指数移动平均?

$$s_t = \rho s_{t-1} + (1 - \rho) g_t^2$$

递推展开:

$$s_t = (1 - \rho) \sum_{i=0}^t \rho^{t-i} g_i^2$$

s_t 是历史**梯度平方**的指数加权移动平均, 它度量了每个参数在"最近一段历史中的**波动程度**". 当某个参数的 s_t 大时 (该参数梯度历史上波动剧烈), $\frac{\eta}{\sqrt{s_t}}$ 自动变小, 起到稳定作用。

RMSProp 的核心价值

问题	RMSProp 如何解决
不同参数梯度尺度差异大	逐参数的自适应学习率 $\eta/\sqrt{s_t}$
损失曲面各方向曲率不同	陡峭方向自动减速, 平缓方向自动加速
训练后期梯度变小	学习率相应放大, 不浪费迭代

1.4 Adam (Adaptive Moment Estimation)

核心理念

Adam 由 Kingma & Ba (2015) 提出，可以看作 **动量法 + RMSProp 的结合体**：

- **一阶矩** m_t (动量)：梯度本身的指数移动平均，提供方向惯性
- **二阶矩** v_t (RMSProp)：梯度平方的指数移动平均，提供逐参数自适应学习率

完整推导

Step 1: 计算一阶矩和二阶矩

$$\begin{aligned}m_t &= \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t \\v_t &= \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2\end{aligned}$$

其中 $\beta_1 = 0.9$ (一阶矩衰减率)， $\beta_2 = 0.999$ (二阶矩衰减率)， $g_t = \nabla_{\theta} \mathcal{L}(\theta_t)$ 。

Step 2: 偏差校正 (Bias Correction)

初始时刻 $m_0 = 0, v_0 = 0$ ，这意味着在训练初期 m_t 和 v_t 会**系统地偏小**（因为初始值为零，加权平均需要时间“预热”）。以 m_t 为例：

$$\begin{aligned}m_1 &= (1 - \beta_1)g_1 \\m_2 &= \beta_1(1 - \beta_1)g_1 + (1 - \beta_1)g_2 \\&= (1 - \beta_1)(\beta_1 g_1 + g_2)\end{aligned}$$

期望 $\mathbb{E}[m_t] = \mathbb{E}[g] \cdot (1 - \beta_1^t) \neq \mathbb{E}[g]$ 。偏差校正将矩估计除以 $(1 - \beta^t)$ ：

$$\begin{aligned}\hat{m}_t &= \frac{m_t}{1 - \beta_1^t} \\\hat{v}_t &= \frac{v_t}{1 - \beta_2^t}\end{aligned}$$

直观理解： $t = 1$ 时 $\beta_1^1 = 0.9$ ，除以 0.1 相当于把初始小值放大 10 倍；随着 $t \rightarrow \infty$ ， $\beta_1^t \rightarrow 0$ ，校正因子 $\rightarrow 1$ ，校正效果自然消失。

Step 3: 参数更新

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \odot \hat{m}_t$$

Adam 的核心优势

- **自适应学习率**： $\frac{1}{\sqrt{\hat{v}_t}}$ 为每个参数独立调整步长
 - **动量方向引导**： $\hat{m}_t$ 作为指数平滑后的梯度方向，噪声小
 - **偏差校正**：训练初期不会因矩估计为零而导致步长异常
 - **超参数鲁棒**：默认 $\beta_1 = 0.9, \beta_2 = 0.999, \eta = 0.001$ 在多数任务上开箱即用
-

1.5 AdamW：解耦权重衰减

问题：Adam 的权重衰减有什么问题？

在原始 Adam 论文中，权重衰减（Weight Decay）是通过 **L2 正则化** 实现的：在损失中加入 $\frac{\lambda}{2} \|\theta\|^2$ 项，则梯度变为 $g_t + \lambda \theta_t$ 。

但在 Adam 中，这个含正则化项的梯度会**进入二阶矩 v_t 的滑动平均**：

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) \cdot (g_t + \lambda \theta_t)^2$$

这导致权重衰减的强度被自适应学习率 $\frac{1}{\sqrt{v_t}}$ **污染**——不同参数的实际衰减强度不同，失去了权重衰减应有的均一正则化效果。

AdamW 的解决方案

Loshchilov & Hutter (2019) 提出 **AdamW**，将权重衰减与梯度更新**解耦**：

$$\begin{aligned} m_t &= \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t \\ v_t &= \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2 \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t} \\ \hat{v}_t &= \frac{v_t}{1 - \beta_2^t} \\ \theta_{t+1} &= \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \odot \hat{m}_t - \underbrace{\eta \cdot \lambda \cdot \theta_t}_{\text{解耦的权重衰减}} \end{aligned}$$

关键区别：

方面	Adam + L2 正则化	AdamW（解耦权重衰减）
权重衰减路径	通过梯度进入 v_t ，被自适应学习率扭曲	独立项 $\eta \lambda \theta_t$ ，不受自适应学习率影响
等效衰减强度	逐参数不同	逐参数均匀
泛化性能	略差	更好（尤其在 Transformer 上）

现代实践：PyTorch 中的 `torch.optim.AdamW` 已成为 Transformer、LLM 训练的默认优化器。HuggingFace Transformers、LLaMA、GPT 系列均使用 AdamW。

1.6 优化器演变总结

$$\begin{aligned} \underbrace{\theta_{t+1} = \theta_t - \eta g_t}_{\text{SGD}} &\longrightarrow \underbrace{v_t = \mu v_{t-1} - \eta g_t}_{\text{Momentum（加惯性）}} \longrightarrow \underbrace{\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{s_t} + \epsilon} g_t}_{\text{RMSProp（逐参数自适应学习率）}} \\ &\longrightarrow \underbrace{\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t}_{\text{Adam（动量 + 自适应学习率 + 偏差校正）}} \longrightarrow \underbrace{\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t - \eta \lambda \theta_t}_{\text{AdamW（解耦权重衰减）}} \end{aligned}$$

优化器	核心创新	适用场景
SGD	基础梯度下降	简单任务、微调阶段
Momentum / NAG	惯性加速 + 方向平滑	峡谷地形、ResNet 训练
RMSProp	逐参数自适应学习率	RNN、非平稳目标
Adam	动量 + 自适应 + 偏差校正	通用首选
AdamW	解耦权重衰减	Transformer / LLM 标配

第二部分：现代激活函数的演变

在第六章中，我们已学习了 Sigmoid、Tanh、ReLU、Leaky ReLU 和 GELU 的基本概念。本节从"为什么需要演变"的角度，系统梳理激活函数从经典到现代的发展脉络，并引入在大语言模型时代至关重要的 SwiGLU。

2.1 回顾：从 Sigmoid 到 ReLU——解决梯度消失

Sigmoid $\sigma(x) = \frac{1}{1+e^{-x}}$ 的致命伤：

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

当 $|x| > 5$ 时， $\sigma'(x) < 0.0066$ ，梯度几乎为零。深层网络中，经过逐层链式相乘，梯度呈**指数衰减**→靠近输入层的参数几乎不更新。

ReLU $\text{ReLU}(x) = \max(0, x)$ 的解法：

$$\text{ReLU}'(x) = \begin{cases} 1, & x > 0 \\ 0, & x \leq 0 \end{cases}$$

正区间梯度恒为 1，彻底消除梯度消失。但代价是 $x \leq 0$ 时梯度为 0，产生 **Dead ReLU** 问题。

2.2 Leaky ReLU：修补负半轴

$$\text{LeakyReLU}(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases} \quad (\text{通常 } \alpha = 0.01)$$

导数：

$$\text{LeakyReLU}'(x) = \begin{cases} 1, & x > 0 \\ \alpha, & x \leq 0 \end{cases}$$

改进点：负半轴保留微小梯度 α ，使 Dead ReLU 的神经元有机会"苏醒"。

局限： α 是固定超参数，缺乏灵活性。由此引出 **PReLU** (Parametric ReLU) —— 让 α 成为可学习参数。

2.3 GELU：引入随机正则化视角

GELU (Gaussian Error Linear Unit) 由 Hendrycks & Gimpel (2016) 提出，是 BERT、GPT-2/3 等 Transformer 模型的标准激活函数。

数学定义

GELU 的核心思想是：**将输入乘以一个概率掩码**——以输入值的大小来决定"通过"的概率：

$$\text{GELU}(x) = x \cdot P(X \leq x), \quad X \sim \mathcal{N}(0, 1)$$

其中 $P(X \leq x) = \Phi(x)$ 是标准正态分布的累积分布函数 (CDF)。

$$\text{GELU}(x) = x \cdot \Phi(x) = x \cdot \frac{1}{2} \left[1 + \text{erf} \left(\frac{x}{\sqrt{2}} \right) \right]$$

近似公式（工程实现）

精确计算 erf 开销大，PyTorch 提供两种近似：

tanh 近似 (PyTorch 默认)：

$$\text{GELU}(x) \approx 0.5x \left[1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right) \right]$$

sigmoid 近似 (QuickGELU)：

$$\text{GELU}(x) \approx x \cdot \sigma(1.702x)$$

导数推导

GELU 的精确导数：

$$\begin{aligned} \text{GELU}'(x) &= \Phi(x) + x \cdot \phi(x) \\ &= \Phi(x) + x \cdot \frac{1}{\sqrt{2\pi}} e^{-x^2/2} \end{aligned}$$

其中 $\phi(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}$ 为标准正态分布的概率密度函数。

直觉：相比 ReLU 的硬边界 ($x > 0$ 开、 $x \leq 0$ 关)，GELU 提供**平滑的概率型门槛**——输入越正通过概率越大，越负通过概率越小，实现软正则化。

GELU vs ReLU 对比

性质	ReLU	GELU
负半轴行为	硬截断为 0	接近 0 但有微小非零输出
原点处可导性	不可导（技术上是次梯度）	处处可导
梯度	正区间恒为 1	平滑变化
正则化	稀疏（硬）	随机（软）
适用场景	CNN、小网络	Transformer、大模型

2.4 SwiGLU：大语言模型时代的选择

为什么需要 GLU 家族

传统激活函数是逐元素的非线性变换 ($\text{ReLU}(Wx + b)$)，它只在一个地方引入非线性。GLU (Gated Linear Unit) 的思路是用门控机制，让网络自己学会哪些信息应该通过、哪些应该被抑制。

GLU 的基本形式

GLU 由 Dauphin et al. (2017) 提出：

$$\text{GLU}(x) = (xW_1 + b_1) \odot \sigma(xW_2 + b_2)$$

两个线性变换后的输出逐元素相乘——一个提供"信号"、一个提供"门控"。门控值在 $(0, 1)$ 之间，控制信号通过的"量"。

类比：GLU 就像 LSTM/GRU 中的门控机制在 FFN（前馈网络）中的应用——输入经过两个并行的线性变换，一个负责内容、一个负责开关。

SwiGLU

Shazeer (2020) 系统研究了 GLU 的各种变体，发现用 **Swish 替代 Sigmoid 作为门控函数** 效果最佳，即 **SwiGLU**：

$$\text{SwiGLU}(x) = \underbrace{(xW_1 + b_1)}_{\text{信号}} \odot \underbrace{\text{Swish}(xW_2 + b_2)}_{\text{门控}}$$

其中 $\text{Swish}(x) = x \cdot \sigma(x)$ （也叫 SiLU——Sigmoid Linear Unit）。

展开形式：

$$\text{SwiGLU}(x) = (xW_1 + b_1) \odot [(xW_2 + b_2) \cdot \sigma(xW_2 + b_2)]$$

注意：由于有两个权重矩阵 W_1 和 W_2 ，SwiGLU 比普通激活函数的参数量多了约 33% ~ 50%（取决于维度分配）。为保持总参数量不变，实践中通常将隐藏维度缩小为原来的 $\frac{2}{3}$ 或做等价的维度调整。

Swish / SiLU 的导数

因为 $\text{Swish}(x) = x \cdot \sigma(x)$ ，对其求导：

$$\begin{aligned}\text{Swish}'(x) &= \sigma(x) + x \cdot \sigma'(x) \\ &= \sigma(x) + x \cdot \sigma(x)(1 - \sigma(x)) \\ &= \sigma(x) [1 + x(1 - \sigma(x))] \\ &= \text{Swish}(x) + \sigma(x)(1 - \text{Swish}(x))\end{aligned}$$

为什么 SwiGLU 好？

- 门控机制：**信号与门控的乘性交互比加性非线性表达能力更强
- Swish 的非单调性：**Swish 在负半轴不是单调下降，而是先降后升再降——这种"非单调弯折"提供了更丰富的梯度模式
- 平滑性：**处处可导，训练更稳定
- 实证优势：**LLaMA、PaLM、Gemma、Qwen 等主流大模型全部使用 SwiGLU

SwiGLU 在 Transformer FFN 中的应用

传统 Transformer FFN (使用 ReLU) :

$$\text{FFN}(x) = W_{\text{out}} \cdot \text{ReLU}(W_{\text{in}} \cdot x + b_{\text{in}}) + b_{\text{out}}$$

SwiGLU FFN (如 LLaMA 中的实现) :

$$\text{FFN}_{\text{SwiGLU}}(x) = W_{\text{out}} \cdot [\text{Swish}(W_{\text{gate}} \cdot x) \odot (W_{\text{in}} \cdot x)]$$

等价于使用 SwiGLU 激活的带门控 FFN。注意三个权重矩阵 W_{gate} , W_{in} , W_{out} 的维度需要协调设计。

第三部分：思考题

思考题 1：为什么激活函数不用 x^2 来引入非线性？

二次函数 $f(x) = x^2$ 显然是**非线性**的，理论上应该能让神经网络摆脱"多层等价于单层"的困境。但实践中没有任何主流网络使用 x^2 作为激活函数。请从梯度行为、数值稳定性、与其他层交互的角度分析原因。

思考题 2：为什么不直接用多项式回归替代"线性层 + 激活函数"的组合？

我们知道 $y = a_n x^n + a_{n-1} x^{n-1} + \dots + a_0$ 也能拟合任意曲线。既然最终目标都是拟合非线性函数，为什么深度学习选择了 $\hat{y} = W_2 \cdot \sigma(W_1 x + b_1) + b_2$ 这种"线性 $\rightarrow$ 激活 $\rightarrow$ 线性"的堆叠方式，而不是直接用高次多项式？请从维度灾难、泛化能力、表示效率等角度分析。

思考题 3：Adam 收敛快但不如 SGD 泛化好——为什么？

Adam 几乎总是比 SGD 收敛得更快，深度学习中几乎没有人用原始 SGD 从头训练大模型。但在一些图像分类任务的**最终精度**对比中，精心调参的 SGD + Momentum 有时反而超过 Adam。更有趣的是，很多精调（Fine-tuning）阶段也偏好 SGD。请从优化器的"探索 vs 收敛"行为差异、以及自适应学习率对泛化的影响角度分析可能的原因。

思考题配套实验

前两道思考题配有可视化实验代码，直观验证理论分析。在 [项目/](#) 目录下运行：

```
cd 第一节课/项目/  
python activation_polynomial_experiments.py
```

将生成两张对比图：

- **图1 (experiment1_x2_activation.png)**: x^2 vs ReLU vs Sigmoid 的梯度行为、符号保留、深层输出分布对比 (6 个子图)
- **图2 (experiment2_poly_vs_nn.png)**: 多项式 vs 神经网络的参数爆炸、Runge 现象、高维泛化对比 (6 个子图)

实验代码纯 NumPy + Matplotlib 实现，无需额外安装依赖，可直接运行。